

軟體工程

第四課：系統塑模與統一架構 (System Modeling & Unified Architecture)

授課教師：薛念林 教授

資訊工程學系

逢甲大學

第四章：課程藍圖與核心架構

第一部分：基礎理論與功能塑模

- **4.1 系統塑模基礎與 UML**：什麼是塑模、四大視角、五大核心圖表、方法論之戰與 UML 三巨頭名片。
- **4.2 系統環境塑模 (Context Diagram)**：Kossiakoff 邊界黑箱原則、外部實體四大分類學 (Wikipedia)、DFD 至 C4 演進，與活動流程。
- **4.3 功能塑模與使用案例**：Jacobson 的歷史遺產、include 與 extend 機制、結構化規格書。

第二部分：結構、動態與 AI 塑模

- **4.4 循序圖與 BCE 架構**：訊息傳遞、生命線與啟用條、邊界-控制-實體 (BCE) 架構模式。
- **4.5 結構塑模與領域類別圖**：領域實體、可見度修飾詞、重數、組合 (Composition) 與聚合 (Aggregation)。
- **4.6 行為塑模與有限狀態機**：訂單生命週期狀態機、事件觸發、守衛條件與轉換動作。
- **4.7 AI 輔助系統塑模**：Text-to-UML 視覺副駕駛、人機協同審查防護、核心填空回顧。

4.1 系統塑模基礎與 UML 演進

"A language that doesn't affect the way you think about programming is not worth knowing."

(一門不能改變你程式思維的語言，就不值得去學。)

— Alan Perlis

”

什麼是系統塑模？ (What is System Modeling?)

"系統塑模是為系統開發抽象模型的過程，每個模型代表該系統不同的視角或觀點。"

— Ian Sommerville, *Software Engineering* (10th ed.)

”

- 系統塑模的核心工程原則：

- **抽象化 (Abstraction)**：隱藏不必要的實作細節，突顯關鍵的架構結構、資料流動與相依關係。
- **多重視角 (Multiple Perspectives)**：沒有任何單一圖表能解釋整個複雜軟體；不同利害關係人需要不同的視角。
- **溝通與驗證 (Communication & Verification)**：作為產品經理、系統架構師、開發者與測試工程師之間的通用視覺語言。
- **實作藍圖 (Blueprint for Implementation)**：提供正式的結構與行為規格，作為後續編寫程式碼與資料庫結構的精確藍圖。

盲人摸象與多重視角模型 (The Parable of the Elephant & Multiple Perspectives)

寓言啟示：盲人摸象與系統塑模的「多重視角」必要性

① 摸到象鼻：「像一條巨蟒 / 水管！」

🎯 互動視角 (Interaction Perspective)

- 關注：訊息傳遞順序、呼叫流向與動態協同
- 盲點：忽視靜態資料結構與內部持久化

對應 UML：循序圖 (Sequence Diagram)

② 摸到象腿：「像一根堅硬巨柱！」

🏠 結構視角 (Structural Perspective)

- 關注：實體資料屬性、關聯、組合與靜態骨幹
- 盲點：看不出系統隨時間演進的動態行為

對應 UML：類別圖 (Class Diagram)

③ 摸到象身：「像一面高大磚牆！」

🛡️ 外部視角 (External / Context)

- 關注：系統邊界、運行環境與外部第三方 API
- 盲點：無法得知內部核心邏輯如何拆解實現

對應 UML：環境圖 (Context Diagram)

④ 摸到象尾：「像一條擺動細繩！」

⚡ 行為視角 (Behavioral Perspective)

- 關注：業務流程流轉、外在事件刺激與狀態轉換
- 盲點：無法單獨反映靜態物件職責與系統邊界

對應 UML：活動圖 / 狀態圖 (Activity / State)

💡 結論：任何單一模型皆為「局部投影」；唯有綜合不同視角，才能完整還原軟體架構全貌！

系統塑模的 4 大核心視角 (4 Core Perspectives)

- **1. 外部視角 (External Perspective) :**
 - 塑模系統處在的運行環境、外部合作夥伴與邊界範圍。
 - 明確劃分系統邊界：哪些由內部團隊開發，哪些委託給外部第三方服務。
- **2. 互動視角 (Interaction Perspective) :**
 - 塑模外部參與者與系統之間，或是內部協同物件之間的動態溝通與訊息傳遞。
- **3. 結構視角 (Structural Perspective) :**
 - 塑模系統資料組織、類別、屬性、方法與關聯性的靜態架構，與執行時間先後無關。
- **4. 行為視角 (Behavioral Perspective) :**
 - 塑模系統的動態執行行為、業務流程步驟，以及系統針對外在事件所產生的反應式狀態轉換。

現代實務必備的 5 大 UML 核心圖表

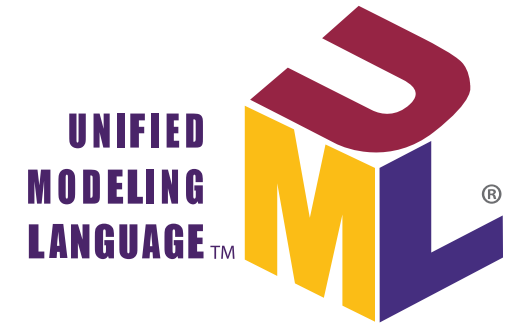
圖表類型	所屬視角	動態 / 靜態	主要軟體工程職責
1. 環境圖 (Context Diagram)	外部視角	靜態邊界	劃定系統邊界與外部夥伴系統的相依關係
2. 活動圖 (Activity Diagram)	行為視角	動態流程	視覺化呈現連續業務工作流程與平行並行邏輯
3. 使用案例圖 (Use Case Diagram)	互動視角	靜態契約	界定使用者目標與系統功能範疇邊界
4. 循序圖 (Sequence Diagram)	互動視角	動態時間	沿著時間生命線追蹤物件之間的循序訊息傳遞
5. 類別圖 (Class Diagram)	結構視角	靜態骨幹	定義領域實體、資料屬性、操作方法與物件關聯
(加分必學) 狀態機圖 (State Diagram)	行為視角	動態反應	塑模離散狀態、事件觸發與反應式生命週期

標準化的迫切需求：1990 年代「方法論之戰」

- 物件導向程式設計的崛起 (1980 年代末至 1990 年代初)：
 - 軟體產業從程序導向程式碼 (C, Pascal) 大規模轉移至物件導向典範 (C++, Smalltalk)。
 - 工程師迫切需要一套圖形符號來視覺化表達類別、物件與關聯。
- 百家爭鳴的「方法論之戰 (Method Wars)」時代：
 - 超過 50 種相互競爭的物件導向塑模記號充斥商業市場。
 - 各派大師激烈爭論：類別究竟該畫成雲朵、矩形還是橢圓？繼承到底該用空心三角、實心箭頭還是虛線？
 - 嚴重的產業割裂碎片化：各公司之間無法交換架構模型，CASE 塑模軟體彼此互不相容，工程師換工作就得被迫重新學習一套新符號。

統一與標準化：從 Rational 到 OMG

- **1994 年 – Rational 展開統一整合：**
 - Jim Rumbaugh 離開奇異 (GE) 加入 Rational Software 與 Grady Booch 攜手，將 Booch 方法與 OMT 融合成「統一方法」(Unified Method v0.8)。
- **1995 年 – 「三巨頭 (Three Amigos)」合體：**
 - Ivar Jacobson 加入 Rational，帶來革命性的使用案例 (Use Case) 與 OOSE 架構思維。
- **1997 年 – OMG 國際標準正式誕生：**
 - 提交給物件管理組織 (OMG)，於 1997 年 11 月獲全票通過，成為全球公認的 **UML 1.1** 國際標準。
- **2005 年 – UML 2.0 架構大改版：**
 - 擴充至 13 種（後續擴增為 14 種）圖表類型，並具備嚴謹的可執行元模型 (Metamodel)。

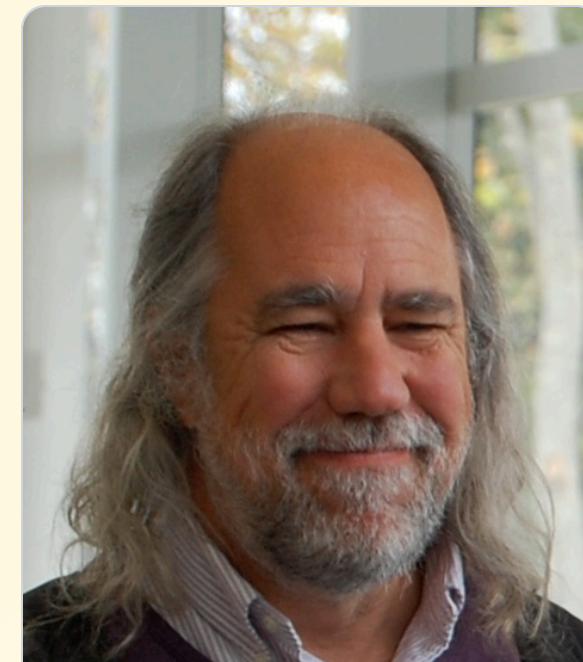


Unified Modeling Language
Object Management Group (OMG)

UML 奠基先驅：Grady Booch

- 角色與學術榮譽：
 - Rational Software 首席科學家、IBM 院士 (Fellow)、ACM 院士。
- 開創經典方法論：
 - **Booch** 方法與劃時代著作：《物件導向分析與設計》(Object-Oriented Analysis and Design with Applications)。
- 對 **UML** 的核心貢獻：
 - 極度專注於具體軟體設計、模組拆解、類別抽象化與架構模式。
 - 提倡為實作層級的物件結構與程式碼對應關係提供高度的視覺表現力。
- 著名軟體工程箴言：

"乾淨的程式碼讀起來，永遠就像是由深具責任心與熱忱的人所撰寫。"



Grady Booch

Rational Software / IBM Fellow

UML 奠基先驅：James Rumbaugh

- 角色與學術榮譽：

- 奇異公司 (GE) 全球研發中心軟體主任研究員、Rational Software、IBM。

- 開創經典方法論：

- 物件塑模技術 (OMT, Object Modeling Technique) 與專書：《物件導向塑模與設計》。

- 對 UML 的核心貢獻：

- 強調嚴謹的領域分析 (Domain Analysis)、語義資料塑模與實體關聯對應。
- 首創將靜態物件結構與 David Harel 的狀態圖 (Statecharts) 深度結合，用以塑模動態反應式系統。

- 著名軟體工程箴言：

“如果不透徹理解領域的真實本質，就絕不可能打造出卓越的軟體。”



James Rumbaugh

GE Research / Rational Software

UML 奠基先驅：Ivar Jacobson

- 角色與學術榮譽：
 - 愛立信 (Ericsson) 首席架構師、Objectory AB 創辦人、Rational Software、SEMAT 發起人。
- 開創經典方法論：
 - 物件導向軟體工程 (OOSE)。
- 對 UML 的核心貢獻：
 - 使用案例 (Use Cases) 的發明者 (1986)：革命性地將系統架構錨定在可衡量的使用者目標之上。
 - 提出經典的邊界-控制-實體 (BCE, Boundary-Control-Entity) 強韌性分析模式與元件化架構。
- 著名軟體工程箴言：

"一個沒有使用者的系統，就沒有任何存在的理由。塑模永遠要先從使用者的目標開始。"



Ivar Jacobson

Ericsson / Objectory / Rational

UML 三巨頭：軟體塑模的三位一體 (The Trinity of Modeling)

Ivar Jacobson

核心回答「Why」(目標 / 使用者)

- 使用案例驅動：外部參與者想達成什麼目標？
- **BCE** 強韌性分析：徹底分離 UI 邊界與核心資料實體。
- 元件契約規範：可重用子系統的架構邊界。

James Rumbaugh

核心回答「What」(領域與資料)

- 領域物件分析：現實世界存在哪些核心概念？
- 類別與資料綱要：結構化的實體關聯關係。
- 有限狀態機：反應式事件驅動的生命週期。

Grady Booch

核心回答「How」(設計與程式碼)

- 物件導向設計：類別之間如何精巧協同運作？
- 巨觀架構拆解：分層模組與相依性控制。
- 實作直譯對應：視覺模型與 OOP 程式碼直接對齊。

📌 **架構整合精髓**：Jacobson 捕捉系統存在的初衷目的 (**Why**)；Rumbaugh 塑模領域存在的資料本質 (**What**)；Booch 設計程式執行的實作架構 (**How**)。

”

觀念檢核測驗 1 (CCQ 1)

在被尊稱為「UML 三巨頭」的軟體工程大師中，哪一位以於 1986 年發明使用案例 (**Use Cases**)、將軟體架構直接錨定於使用者具體目標而聞名？

- **A.** Grady Booch
- **B.** James Rumbaugh
- **C.** Ivar Jacobson
- **D.** Martin Fowler



4.2 系統環境塑模 (System Context Diagram)

"Such a diagram pictures the system at the center, with no details of its interior structure, surrounded by all its interacting systems, environments and activities."

(環境圖將系統置於正中央，不展示任何內部結構細節，環繞著所有與之互動的系統、環境與活動。)

— Alexander Kossiakoff & William Sweet (*Systems Engineering Principles and Practice*)

”

什麼是系統環境圖？ (What is a System Context Diagram?)

"A system context diagram defines the boundary between the system, or part of a system, and its environment, showing the entities that interact with it." — Wikipedia / Systems Engineering Standard „

III 定義與黑箱原則 (Black-Box Principle)

- **最高層級鳥瞰 (Highest-Level View):** 軟體架構在最頂層的整體全景圖，類似方塊圖 (Block Diagram)。
- **嚴格黑箱封裝 (Strict Black-Box):** 將目標系統置於正中央，完全不展示內部結構、模組或程式碼細節。
- **直白共通語言 (Plain Language):** 專為所有利害關係人 (技術架構師、PM、客戶、業務、法務) 設計，無技術門檻。

II 架構工程的兩大核心使命

- **1. 早期範疇共識 (Scope Agreement):** 嚴格劃定 **In-Scope (我方構建)** 與 **Out-of-Scope (外部現成)**，從源頭杜絕**「範疇蔓延 (Scope Creep)」**。
- **2. 盤點外部相依與約束 (Dependencies & Constraints):** 及早識別第三方 API、SLA 服務水準、網路通訊協定與法規約束 (如 PCI-DSS/GDPR)。

系統環境圖的三大基本構件 (Core Building Blocks)

✚ 極簡而強大的雙構件架構：依據系統工程規範，任何環境圖皆由「實體」與「關係」三大核心要素構成。

1. 實體 (Entities / Actors)

- 中央目標系統 (Central System):
 - 置於圖表正中央，代表本次工程專案負責建置的目標軟體本體。
 - 通常以單一醒目的矩形、方塊或圓形表示。
- 外部實體 (External Entities / Terminators):
 - 環繞在中央系統周圍的所有外部人、事、物。
 - 包含終端使用者（火柴人）、外部企業系統、雲端服務、硬體感測器或外部資料庫。

2. 關係與資料流 (Relationships & Flows)

- 具名箭頭線段 (Labeled Directed Links):
 - 連接外部實體與中央系統，明確標示互動與資料傳遞方向。
- 業務語意導向 (Business Semantics):
 - 箭頭旁必須標註具體業務動詞或資料名稱（例如：「送出外送訂單」、「請求金流預授權」、「回傳 GPS 座標」）。
 - 強調傳遞的資訊流 (Information Flow)，而非底層程式呼叫細節。

外部實體分類學：四大互動類型 (Wikipedia Taxonomy)

🕒 精準識別外部邊界：維基百科將環境圖中的外部實體劃分為四種經典角色，有助於評估相依性與風險。

⚡ 1. 主動型實體 (Active Entities)

- 定義：動態發起互動以達成特定業務目標的角色。
- 實例：App 終端顧客、外送司機、合作餐廳店長。
- 特徵：互動頻率高，驅動核心業務交易流程。

🛡️ 2. 被動型實體 (Passive Entities)

- 定義：靜態且低頻與系統互動的維運或稽核實體。
- 實例：系統管理員、資料庫 DBA、資安稽核員。
- 特徵：偏向後台維護、參數配置與定期合規稽核。

🤝 3. 協力型實體 (Cooperative Entities)

- 定義：具可預期行為、被系統調用以實現業務目標的外部夥伴。
- 實例：Stripe 金流閘道、Google Maps 地圖 API、Firebase。
- 特徵：需簽署 SLA 服務協議，需設計斷線降級容錯。

⚖️ 4. 自主約束型實體 (Autonomous Entities)

- 定義：獨立於系統之外，但透過規範法規施加邊界限制的實體。
- 實例：金融金管會、PCI-DSS 安全標準、GDPR 隱私法。
- 特徵：不直接調用 API，但強烈約束系統的架構邊界。

環境圖的演進與跨方法論對照 (Evolution & Alternatives)

🔄 從結構化分析到現代微服務架構：環境模型歷經數十年演進，始終是架構設計的基石。

”

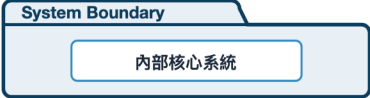
📖 歷史演進與跨學派實踐

- 結構化分析 (DFD Level 0) :
 - Edward Yourdon 與 Tom DeMarco 於 1970 年代首創。以單一「0 號處理泡泡 (Bubble 0)」代表整個系統。
- UML 的跨視角實踐：
 - UML 雖未單獨設立 "Context Diagram" 名稱，但實務上以**「帶系統邊界框的高階使用案例圖」或「高階構件圖」**實踐。
- 現代 C4 Model (Simon Brown) :
 - 現代雲架構標準，**Level 1 就是 System Context Diagram**，成為微服務溝通的第一共通語言。

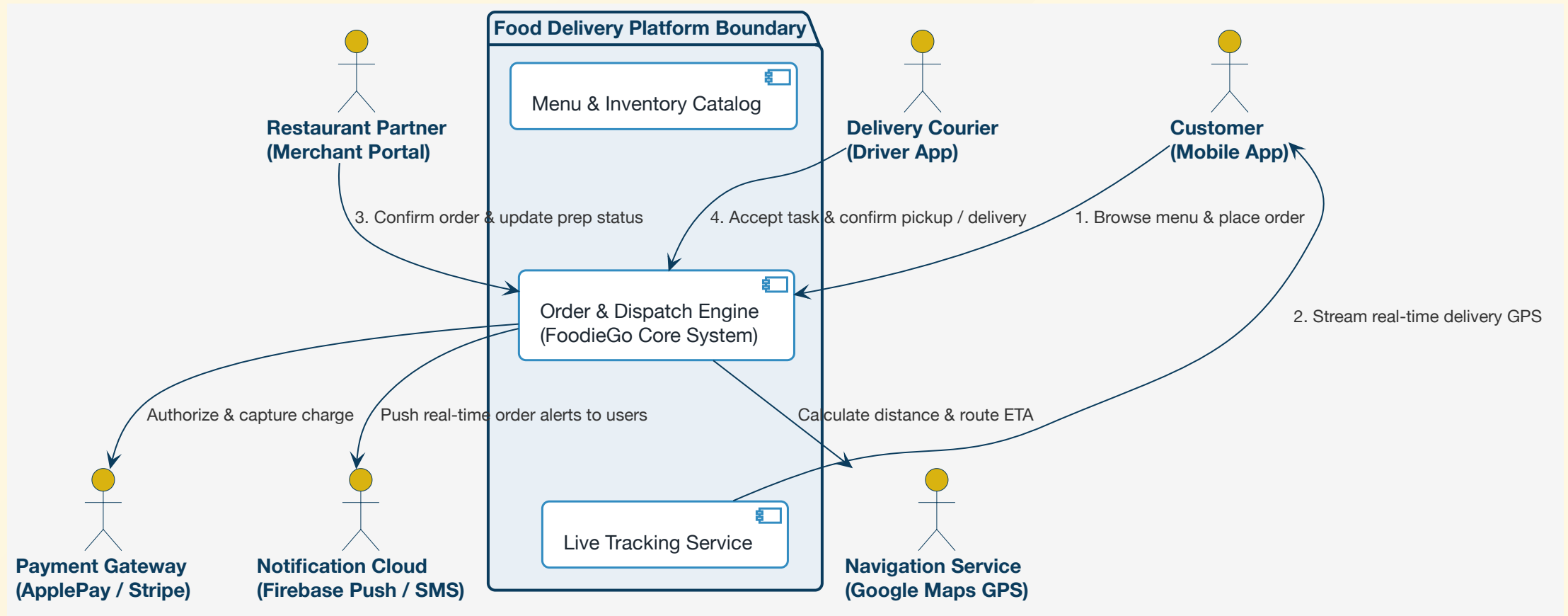
📏 設計黃金準則 (Rule of Thumb)

- 連線數量控制 (Wikipedia 準則) :
 - 外部連線應維持在 **15 到 20 條以內**。
 - 若連線過多 (>20)，代表抽象層次不足，圖形將淪為難以辨識的「義大利麵連線」，應適度分組或模組化。
- 保持黑箱邊界：
 - 嚴格克制「想把內部資料庫或微服務全畫出來」的衝動，保持頂層架構邊界的純粹性。

系統環境圖符號與語意對照 (Context Model Notation)

系統環境圖 (System Context Diagram)：核心符號圖例與語意對照表		
視覺符號 (Visual Notation)	語法名稱 (Element Name)	軟體工程語意與實務職責 (Engineering Semantics)
	系統架構邊界 System Boundary Box	• 確立範疇 (In-Scope)：框內為我方團隊自主設計、開發與維運的程式碼與微服務。 • 框外為外部人類角色與第三方服務，明確劃分法律責任、成本核算與防範範疇蔓延 (Scope Creep)。
	外部人類參與者 Human Actor (User)	• 真實使用者角色：與系統具備明確互動介面的人類角色（顧客、外送員、餐廳商家）。 • 通常透過特定客戶端終端（如 iOS/Android 原生 App 或商家 Web 後台）與核心系統溝通。
	外部系統 / 雲端服務 External System / Cloud API	• 第三方現成能力：非團隊自主開發的外部現成服務（如 Stripe 金流、Google Maps、Firebase 等）。 • 透過 Web API / Webhook 介接；需早期評估 SLA 可用性、計費成本與合規隔離（如 PCI-DSS）。
	內部核心構件 / 服務 Internal Component / Service	• 核心業務邏輯載體：系統邊界內承載業務規則的核心微服務（如目錄服務、訂單引擎、即時配送引擎等）。 • 右上角帶有 UML 構件符號（小插槽箱），表示其為可獨立部署與維護的內部模組單元。
	相依與資料通訊關聯 Communication Dependency	• 通訊與依賴方向：箭頭方向代表請求發起端或資料相依流向，明確標示誰呼叫誰。 • 線條附帶具名動詞短語與序號，清楚描述該通道所承載的具體商業活動或資料流。

美食外送平台：系統環境模型實戰 (Case Study: Food Delivery Context Model)



美食外送環境圖深度解析：邊界劃定與實體對映

● **營運邊界與實體對映：** 運用外部實體分類學，深度剖析美食外送平台的架構權責劃分。

”

📍 四大實體角色落地對映

- **主動型實體 (Active)：**
 - **顧客 (App):** 瀏覽菜單、下單與即時追蹤外送。
 - **外送員 (App):** 接收派單任務、回傳即時 GPS 座標。
 - **餐廳 (Web/平板):** 接單出餐、更新備餐進度。
- **協力型實體 (Cooperative Cloud):**
 - **Stripe / Apple Pay:** 委外金融授權扣款。
 - **Google Maps:** 路線距離幾何與外送員 ETA 運算。
 - **Firebase:** 跨平台非同步推播與 SMS 驗證碼。

● 架構邊界效益與 SLA 容錯

- **自主約束與資安隔離 (PCI-DSS):**
 - 金融法規強迫扣款行為外包，使內部資料庫完全排除在昂貴的 PCI-DSS 稽核之外。
- **明確責任歸屬 (Anti-Scope Creep):**
 - 內部團隊專注於核心訂單引擎與派單調度演算法，不重複造輪子。
- **外部依賴降級防護 (SLA Fallback):**
 - 當 Google Maps 斷線或 Firebase 延遲時，系統應具備直線距離估算與輪詢備援機制。

從環境邊界邁向業務流程：UML 活動圖 (Bridging to Process)

 由「外在邊界」深入「內部協同」：環境圖定義了「誰在邊界外」，活動圖則接棒展開「跨角色的端到端動態流程」。

”

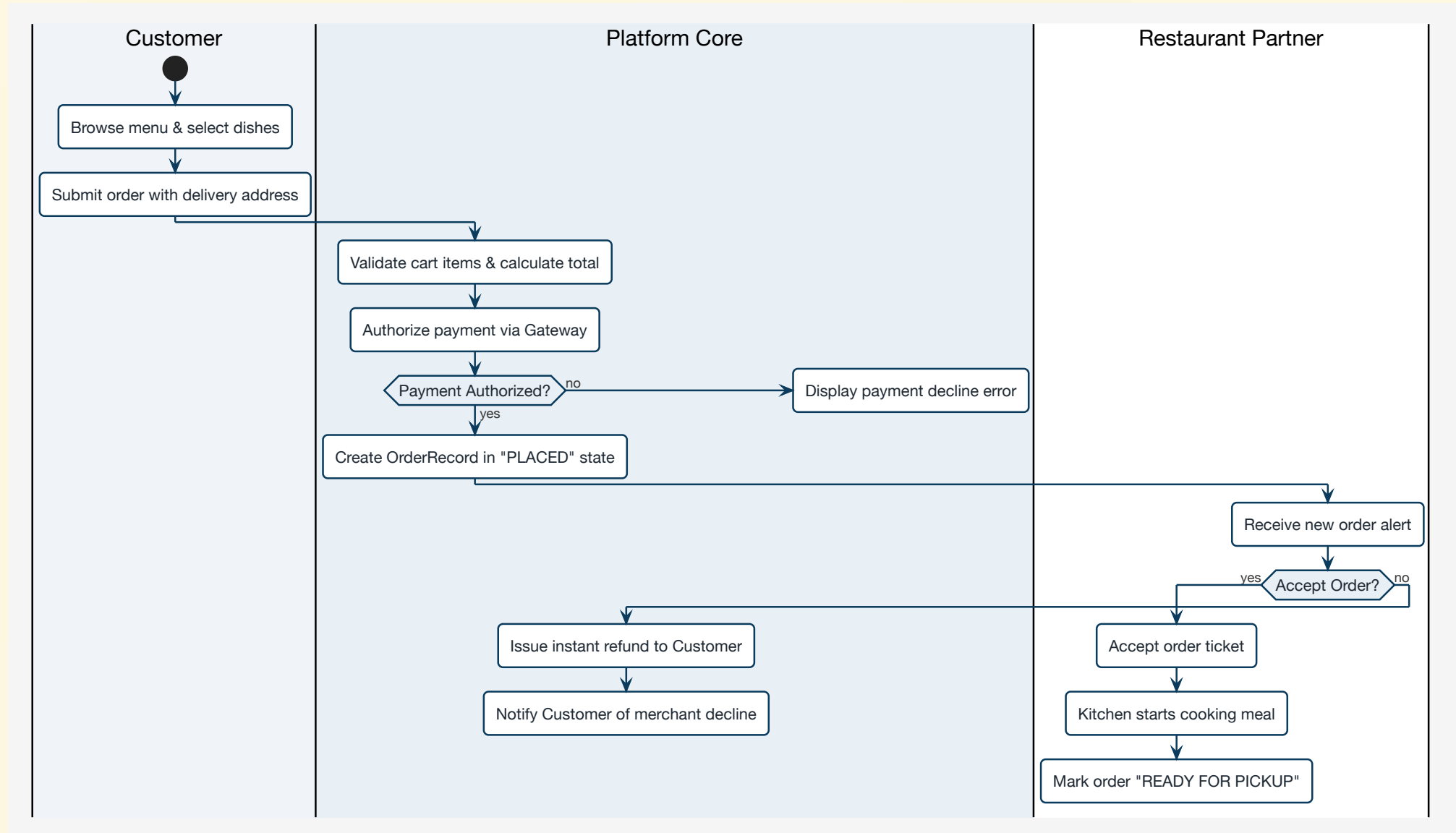
1. 環境圖 (Context Diagram)

- 視角：外部靜態邊界 (Where & Who)。
- 粒度：系統最高層級黑箱視角。
- 職責：
 - 確立責任邊界，防範範疇蔓延。
 - 識別外部主動、被動、協力與自主約束實體。
 - 盤點跨組織網路 API 與相依合約。

2. 活動圖 (Activity Diagram)

- 視角：動態業務程序 (How & When)。
- 粒度：跨泳道步驟與分支判斷細節。
- 職責：
 - 以泳道 (Swimlanes) 分配不同角色的執行步驟。
 - 描述循序動作、條件分支、平行並行與異常終止。
 - 串聯顧客、平台與餐廳的真實業務生命週期。

美食外送履行：顧客下單與廚房備餐活動流程



第一階段活動流程與語法符號解析

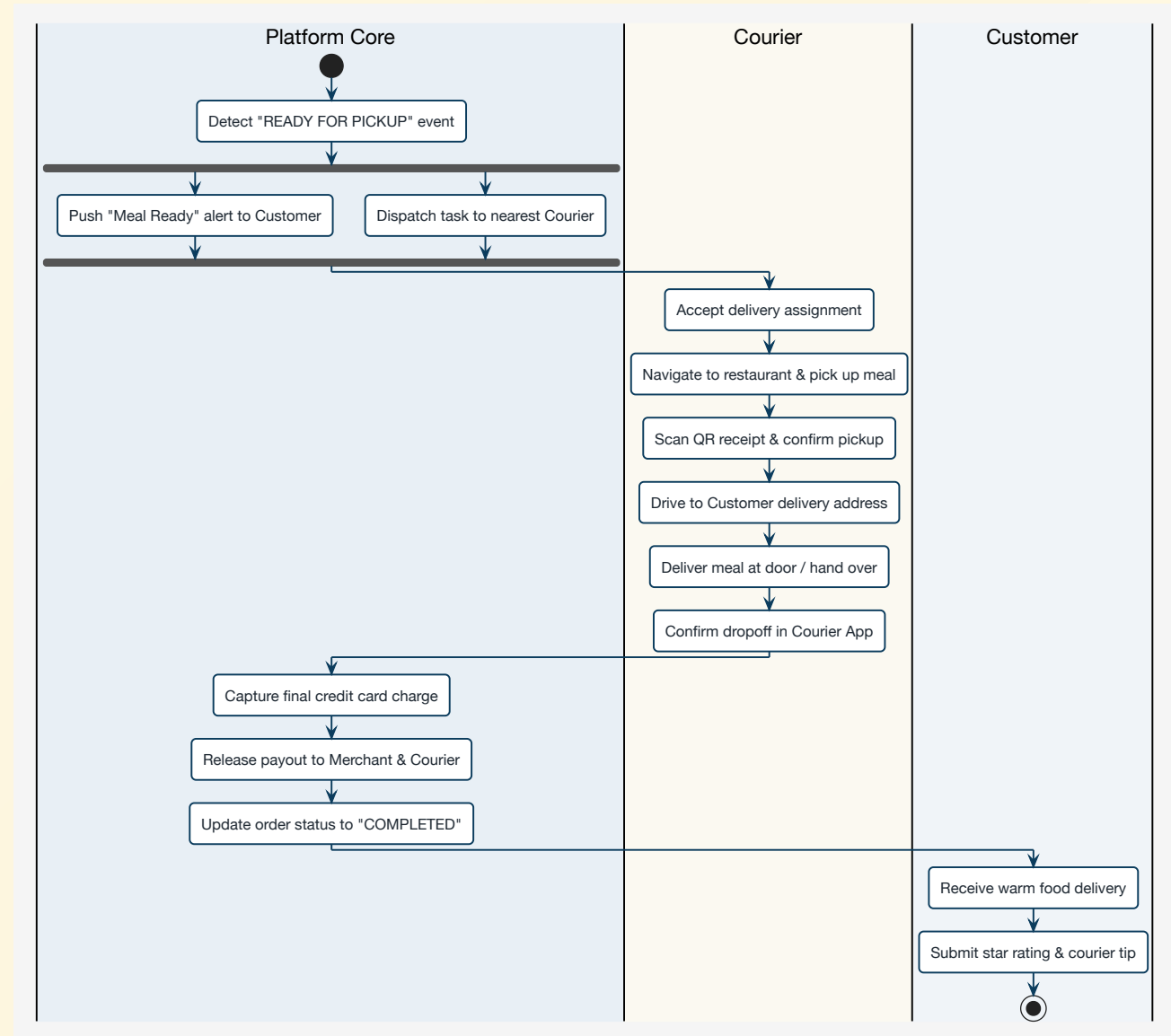
- 活動塑模的核心語法元素：

- 圓角矩形 (Rounded Rectangles)：操作動作狀態（例如 瀏覽菜單並挑選餐點 、 廚房開始烹調餐點 ）。
- 泳道分區 (Swimlanes / Partitions)：將工作職責明確分配給顧客、平台核心與餐廳夥伴。
- 決策菱形 (Decision Diamonds)：依據布林守衛條件進行分支判斷（ [授權扣款成功?] 、 [餐廳是否接單?] ）。
- 中斷終止節點 (Detach Node / Terminal Exit)：代表異常分支終止（例如付款失敗或餐廳原物料短缺拒單）。

- 非同步解耦的工程考量：

- 顧客的互動在完成訂單提交後即告一段落；後續步驟由平台與商家非同步推進。
- 當餐廳拒單時，平台核心能自動觸發即時退款與發票作廢，維護商業信用。

美食外送履行：外送派單與實體交付活動流程



第二階段活動流程：並行並發與分岔 / 匯合機制

- **分岔與匯合同步棒 (Fork & Join Bars，粗水平線)：**
 - **分岔棒 (Fork Bar，並發產生器)：** 將單一控制執行緒拆分為多個平行並行的非同步活動。
 - **實例：** 平台同時發送推播通知給顧客，並平行啟動 GPS 演算派單給外送員，互不卡住阻斷！
 - **匯合棒 (Join Bar，同步屏障)：** 匯總多個平行並發分支；只有當所有流入的平行前置任務全部完成後，後續流程才會繼續。
- **實體世界履行的工程防護設計：**
 - **虛實整合握手驗證：** 外送員抵達餐廳後，必須透過手機鏡頭掃描訂單 QR Code，原子化確認餐點取走。
 - **資金正式清算觸發點：** 信用卡的最終正式扣款 (Capture) 與平台抽成清算，*僅在實體驗收交付完成時正式觸發。*
 - **顧客回饋閉環：** 完成送達後立即推播評分與給予小費介面。

4.3 功能塑模與使用案例模型

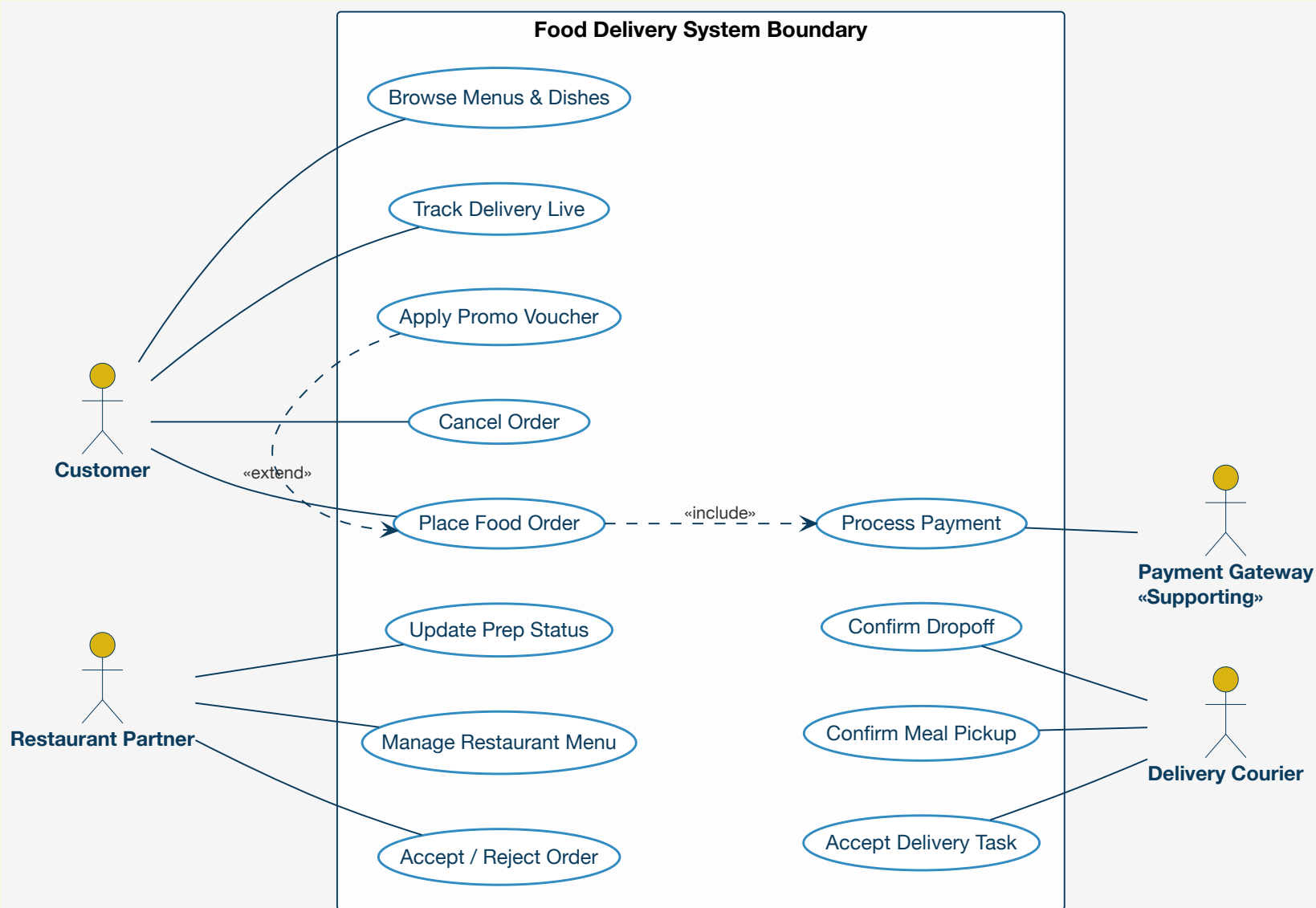
"A use case is a contract for behavior between the system and its actors to achieve a measurable business goal."

(使用案例是系統與參與者之間的行為契約，旨在達成可衡量的商業目標。)

— Alistair Cockburn

”

美食外送平台：使用案例圖 (Use Case Diagram)



使用案例模型與參與者角色解析

- 主要參與者 vs. 支援輔助參與者：

- 主要參與者 (**Primary Actors**，顧客、餐廳、外送員)：主動發起使用案例，以達成個人的商業目標（例如享用美食、營收獲利、賺取配送費）。
- 次要 / 支援參與者 (**Supporting / Secondary Actors**，金流閘道)：由系統主動呼叫以協助完成使用案例之外部系統（例如進行信用卡扣款與反洗錢驗證）。

- 系統邊界劃分原則：

- 所有使用案例橢圓一律位於矩形邊界**內部**；所有參與者一律位於邊界**外部**。
- 連接線代表參與者參與該項功能互動。

- 使用案例粒度 (**Granularity**) 黃金法則：

- 使用案例必須代表一項**完整的**、**能為使用者帶來獨立商業價值**的交易。
- **反面教材**：「輸入密碼」、「點擊送出按鈕」（這些只是微小的 UI 操作，絕非使用案例！）。
- **正面典範**：「美食訂餐下單」、「管理餐廳菜單」（能交付完整的商業成果）。

符號深解： <<include>> 與 <<extend>> 的關鍵差異

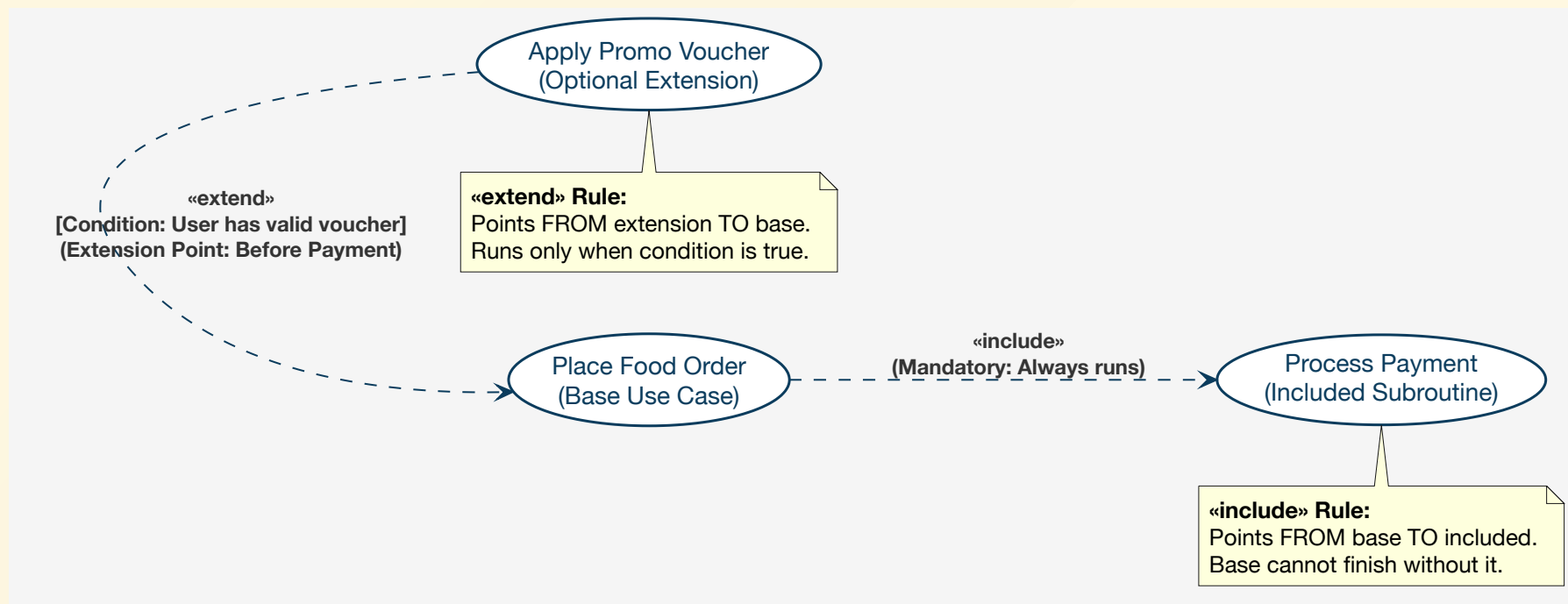
<<include>> (必要包含)

- **核心語義：** 基礎使用案例在執行過程中，必須且一定會執行被包含的子程序。
- **箭頭方向：** 由基礎案例指向被包含案例 (`Base ..> Included`)。
- **軟體工程目的：** 將多個案例共用的必要重複邏輯抽離成獨立子案例（例如：多種訂單與會員訂閱流程均強制需要處理信用卡扣款）。

<<extend>> (條件擴充)

- **核心語義：** 擴充行為僅在特定條件成立時，才會外加插入基礎流程之中。
- **箭頭方向：** 由擴充案例指向基礎案例 (`Extension ..> Base`)。
- **軟體工程目的：** 將偶發的選用邊界情境（如套用折扣優惠碼、無接觸配送要求）完全隔離，避免弄髒主要的正常流程。

Include 與 Extend：視覺語法與箭頭方向機制



結構化使用案例規格書：美食訂餐下單 (UC-01)

規格書欄位	技術規格描述與工程契約
使用案例編號與名稱	UC-01: 美食訂餐下單 (Place Food Order)
主要參與者	顧客 (已註冊登入之美食外送 App 會員)
前置條件 (Preconditions)	顧客通過驗證登入；購物車內含有 ≥ 1 件營業中合作商家之餐點。
後置條件 (Postconditions)	訂單狀態標記為 PAID ；通知廚房接單；外送排程佇列；收據寄發顧客信箱。
主要成功路徑 (Happy Path)	- 顧客於結帳確認畫面點擊「確認付款下單」。 - 系統即時驗證餐點供應庫存，並計算餐點總額、稅金、外送費與服務費。 - 系統執行 <<include>> UC-02: 處理信用卡扣款。 - 系統在資料庫中持久化建立具唯一 orderId 之訂單實體。 - 系統傳送即時訂單明細至餐廳夥伴之後台平板。 - 系統建立即時 GPS 追蹤作業並於顧客端顯示預估抵達時間。
擴充與替代路徑	3a. 信用卡授權遭拒： 3a1. 系統通知顧客卡片扣款失敗；訂單保留於 DRAFT 草稿狀態。 4a. <<extend>> 套用優惠折扣碼： 4a1. 顧客輸入有效折扣碼，系統於步驟 3 扣款前重新計算折抵金額。

觀念檢核測驗 2 (CCQ 2)

在美食外送使用案例模型中，為何「套用優惠折扣碼」使用 `<<extend>>` 指向「美食訂餐下單」，而「美食訂餐下單」卻使用 `<<include>>` 指向「處理信用卡扣款」？

- **A.** 折扣碼是每筆訂單強制必填的欄位；而扣款則是自由選填的項目。
- **B.** 折扣碼為特定條件下的選用行為；而扣款則是結帳必定執行的共用程序。
- **C.** 折扣碼由次要參與者發起執行；而扣款則由主要參與者獨立完成。
- **D.** 折扣碼代表類別層級的繼承關係；而扣款代表物件層級的組合關係。



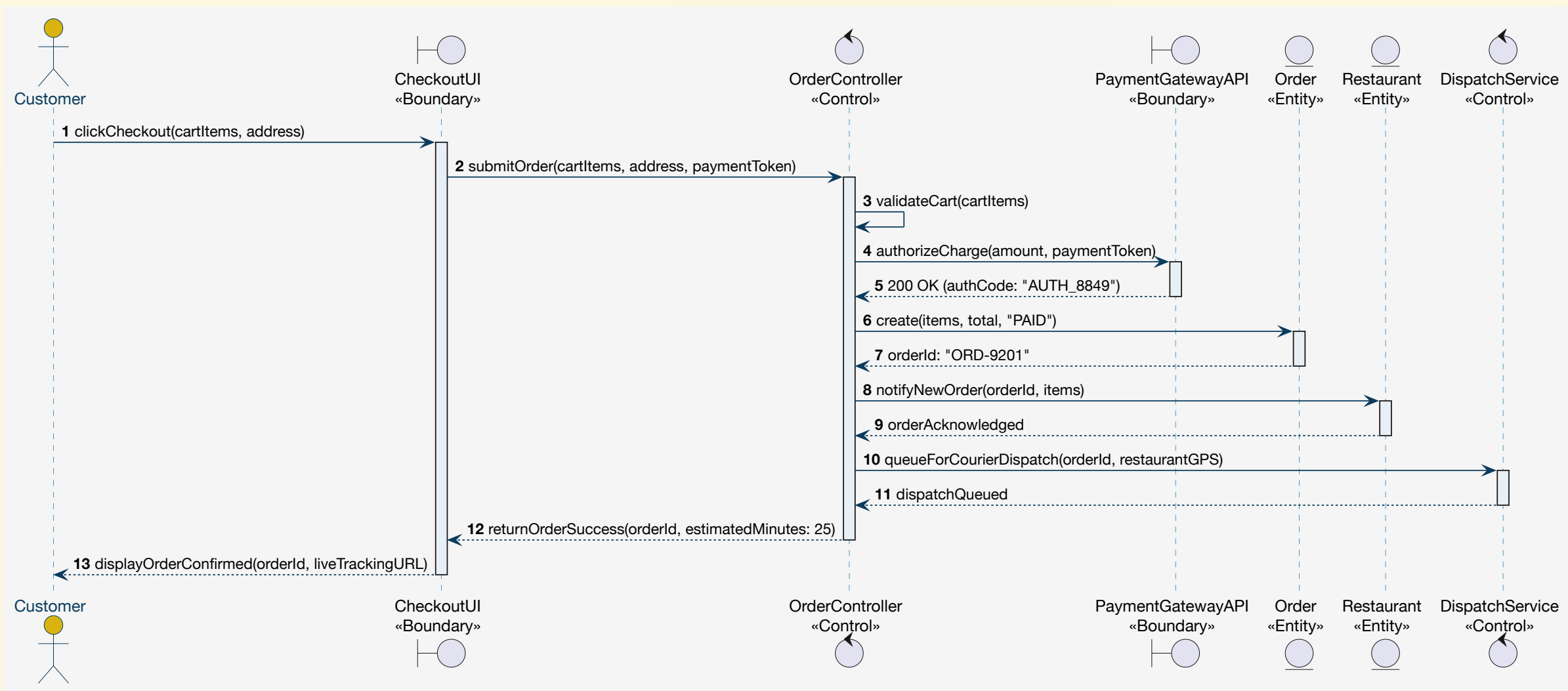
4.4 循序圖與 BCE 架構

"Interaction modeling shows how objects collaborate over time to fulfill the promise of a use case."

(互動塑模展示了物件如何在時間維度上協同運作，以實現使用案例的承諾。)

”

訂單下單與支付流程：UML 循序圖



循序訊息流動與 BCE 架構模式剖析

- 邊界-控制-實體 (BCE, Boundary-Control-Entity) 架構模式：

- 邊界物件 (`<<Boundary>>`)：負責與外部人類參與者及第三方 API 溝通（如 `CheckoutUI`、`PaymentGatewayAPI`）。
- 控制物件 (`<<Control>>`)：協調交易流程、業務邏輯演算法與流程分派（如 `OrderController`、`DispatchService`）。
- 實體物件 (`<<Entity>>`)：封裝持久化儲存的領域狀態與業務核心資料（如 `Order`、`Restaurant`）。

- BCE 架構不可動搖的黃金鐵律：

- 外部使用者與 UI 邊界物件絕不可直接碰觸或存取 Entity 資料實體物件！
- 呼叫流向必須嚴格遵守：**Actor** → **Boundary** → **Control** → **Entity**。
- 原因：徹底實現介面與資料綱要的解耦。未來資料庫欄位或 ORM 變更時，UI 畫面完全不受波及。

循序圖符號與語意指引

視覺符號	語意符號名稱	精確技術語義
垂直虛線	生命線 (Lifeline)	代表該物件實例在記憶體中隨時間推移的存續狀態。
狹長垂直矩形	啟用條 (Activation Bar)	代表該物件實例正在 CPU 上積極執行程式碼運算的時間區間。
實線 + 實心箭頭	同步呼叫 (Synchronous Call)	阻斷性呼叫 (Blocking)；呼叫端暫停等待回傳結果後方能繼續。
實線 + 開放箭頭	非同步訊息 (Asynchronous Call)	非阻斷性訊息 (Non-blocking)；呼叫端送出訊息後立即繼續執行。
虛線 + 開放箭頭	回傳訊息 (Return Message)	將運算結果或資料實例明確回傳給原始呼叫者。
自我迴圈箭頭	自身調用 (Self-Invocation)	物件呼叫自身的私有方法（例如 <code>validateCart()</code> ）。

 **幾何黃金法則：** 時間沿著垂直軸嚴格向下推進。任何訊息箭頭絕對不可向上逆行！

”

觀念檢核測驗 3 (CCQ 3)

在採用 邊界-控制-實體 (BCE) 架構設計的 UML 循序圖中，當使用者於 `CheckoutUI` 點擊結帳送出時，該請求應該直接交由哪一個物件接收處理？

- A. 直接交給 `Order` 實體物件，以便第一時間將訂單寫入資料庫。
- B. 直接交給 `PaymentGatewayAPI` 邊界物件，以便立刻進行信用卡扣款。
- C. 交給 `OrderController` 控制物件，以統籌驗證業務規則與協調交易。
- D. 直接交給 `Restaurant` 實體物件，以確認廚房產能是否充足。



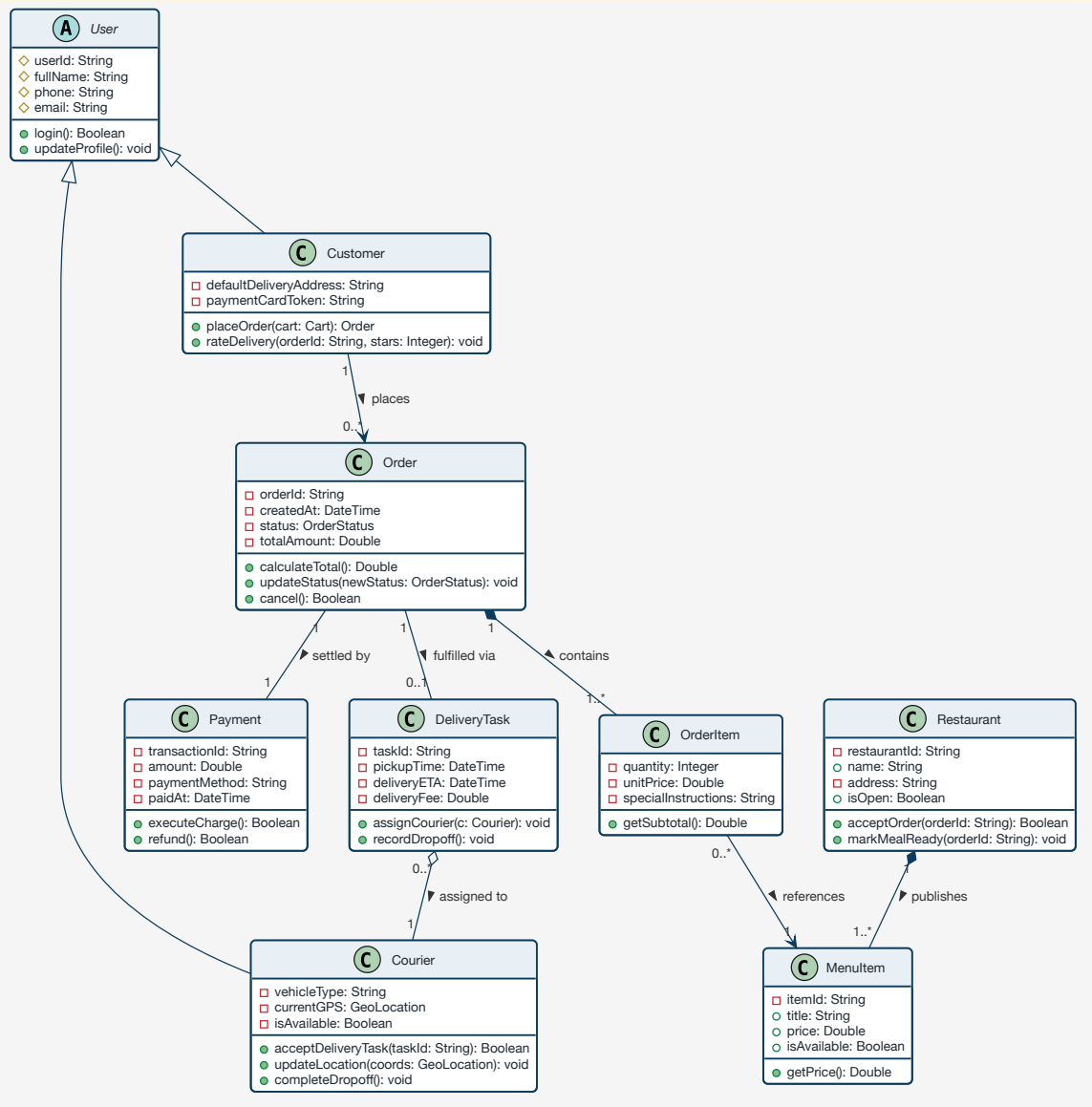
4.5 結構塑模與領域類別圖

"Classes are the static building blocks; objects are the living runtime instances."

(類別是系統的靜態積木；物件是運作時的鮮活實例。)

”

美食外送領域：類別圖架構藍圖



領域類別結構與物件生命週期關聯解析

• 泛化繼承階層 (Generalization / Inheritance) :

- `User` 為抽象父類別，定義共用屬性 (`userId`, `phone`, `email`) 與方法 (`login()`)。
- `Customer` 與 `Courier` 特化繼承 `User`，共享帳號身分同時擴充專屬欄位 (如送餐地址 vs. 載具型態與 GPS 座標)。

• 組合 (◼ 實心菱形, Composition) — 強整體-部分擁有關係 :

- `Order "1" *-- "1..*" OrderItem` : `OrderItem` (例如兩份辣味漢堡) 脫離父層 `Order` 便無獨立存在的意義。若訂單被刪除，其下的訂單明細必須連帶被串聯刪除 (Cascade Delete) ！
- `Restaurant "1" *-- "1..*" MenuItem` : 菜單料理嚴格附屬於該發布餐廳。

• 聚合 (◊ 空心菱形, Aggregation) — 弱整體-部分擁有關係 :

- `DeliveryTask "0..*" o-- "1" Courier` : 外送任務分派給外送員，但外送員具備完全獨立的生命週期。外送任務結束或被取消時，外送員依然留在系統中！

• 關聯與價格歷史快照去耦合 :

- `OrderItem` 以 `0..* --> 1` 關聯 `MenuItem`。 `OrderItem` 獨立記錄下單當下的歷史成交單價，未來餐廳調漲漢堡價格時，過去的歷史訂單金額絕不會受到波及。

類別圖標準語法：三格矩形、可見度與關聯重數

- 標準三格類別方塊 (Three-Compartment Box) :
 - 頂層方格：類別名稱，採 `PascalCase`（斜體代表 `abstract` 抽象類別）。
 - 中層方格：屬性欄位： `[可見度] 名稱 : 型別 [= 預設值]` 。
 - 底層方格：操作方法： `[可見度] 名稱 (參數 : 型別) : 回傳型別` 。
- 可見度修飾詞 (Visibility Modifiers，封裝文法) :
 - `+` **Public (公有)**：整個系統任何類別皆可公開存取。
 - `-` **Private (私有)**：嚴格封裝於本類別宣告內部。
 - `#` **Protected (保護)**：僅限本類別與其衍生子類別存取。
 - `~` **Package (套件私有)**：僅限同模組或命名空間內存取。
- 關聯端重數標示 (Multiplicity) :
 - `1`：剛好精確 1 個實例。
 - `0..1`：選用；0 個或 1 個實例。
 - `0..*` (或 `*`)：0 個至多個實例。
 - `1..*`：至少 1 個實例（上不封頂）。

觀念檢核測驗 4 (CCQ 4)

在美食外送類別圖中，為何「訂單 (Order)」與「訂單明細 (OrderItem)」之間採用組合 (◆)，而「外送任務 (DeliveryTask)」與「外送員 (Courier)」之間卻採用聚合 (◇)？

- **A.** 訂單明細可脫離訂單獨立存在；但外送員失去任務就無法存活。
- **B.** 訂單明細隨訂單銷毀而連帶消失；外送員則具備完全獨立的生命週期。
- **C.** 組合代表類別層級的繼承；聚合代表執行時的方法調用。
- **D.** 組合強制規定零對一重數；聚合強制規定一對多重數。



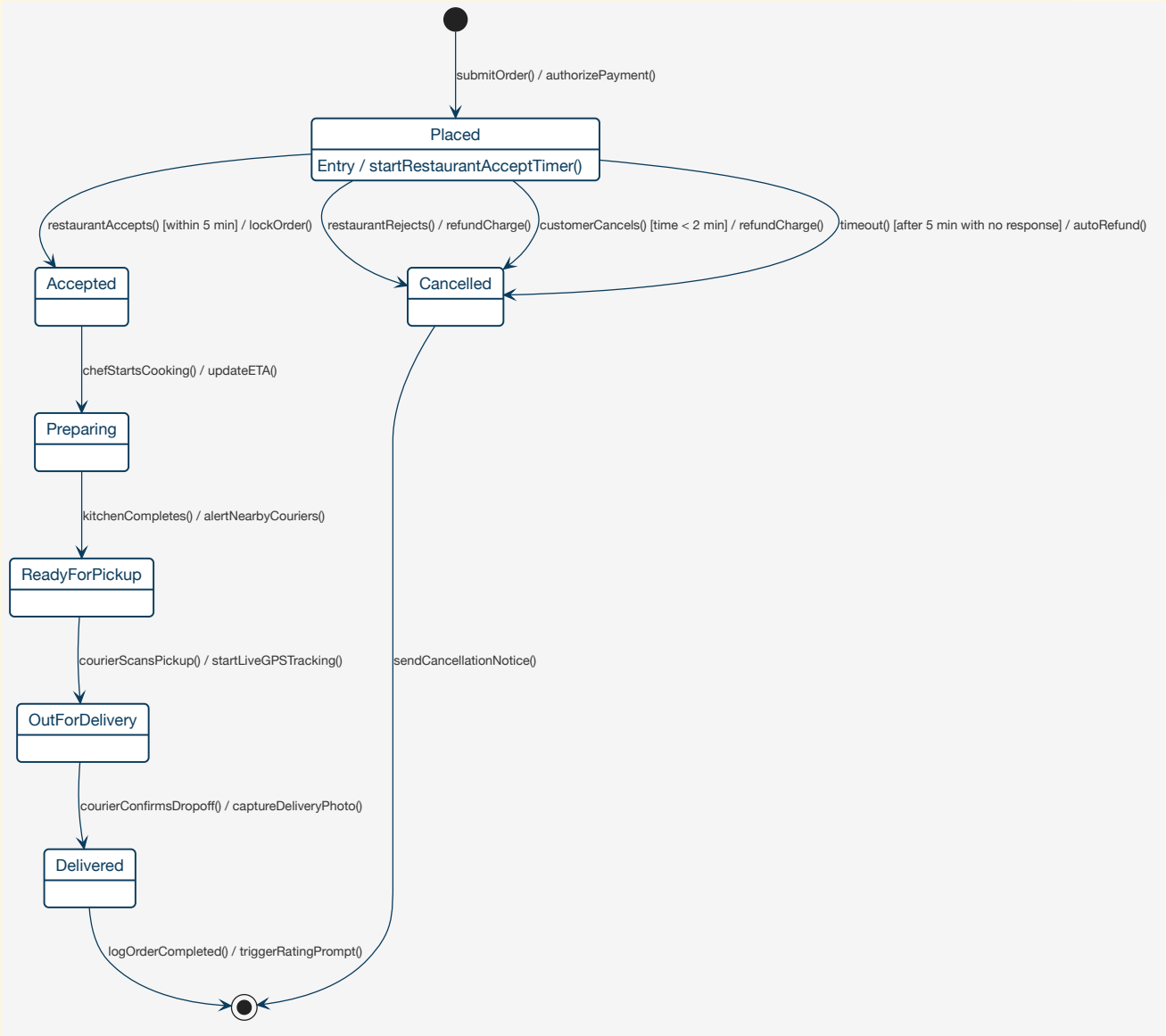
4.6 行為塑模與有限狀態機模型

"A system in dynamic execution is defined by the states it occupies and the events that trigger transitions."

(動態執行中的系統，是由其所處的狀態以及觸發狀態轉換的事件所定義。)

”

訂單生命週期：UML 狀態機圖 (State Machine Diagram)



狀態圖運作原理與轉換文法解析

- 有限狀態機 (FSM) 核心原理：

- 在任何單一運行瞬間，一個 `Order` 物件實例恰好且只能處於一個離散狀態。
- 訂單對外界事件的反應，完全取決於其當前所處的活躍狀態。

- 標準狀態轉換標籤文法 (Formal Label Grammar)：

觸發事件 (Event) [守衛條件 (Guard)] / 執行動作 (Action)

- 觸發事件 (Trigger Event)：刺激狀態轉移的外部或內部事件（如 `chefStartsCooking()` , `courierScansPickup()`）。
- 守衛條件 ([...] Guard Condition)：必須計算為 `true` 轉換才會被放行的布林條件（如 `[within 5 min]` , `[time < 2 min]`）。
- 執行動作 (/ ... Action Effect)：狀態轉移瞬間原子化執行的計算動作（如 `/ refundCharge()` , `/ startLiveGPSTracking()`）。

- 狀態進入動作 (State Entry Actions)：

- 進入特定狀態時自動執行的動作（例如：`Placed: Entry / startRestaurantAcceptTimer()`）。

觀念檢核測驗 5 (CCQ 5)

在訂單生命週期狀態機中，標籤

`customerCancels()` `[time < 2 min]` / `refundCharge()` 各自代表何種技術語義？

- **A.** `customerCancels()` 為守衛條件； `[time < 2 min]` 為觸發事件；
`refundCharge()` 為目標狀態。
- **B.** `customerCancels()` 為觸發事件； `[time < 2 min]` 為布林守衛條件；
`refundCharge()` 為轉換執行動作。
- **C.** `customerCancels()` 為目標類別； `[time < 2 min]` 為呼叫方法；
`refundCharge()` 為傳回型別。
- **D.** `customerCancels()` 為主要參與者； `[time < 2 min]` 為逾時限制；
`refundCharge()` 為物件生命線。



4.7 AI 輔助系統塑模

"AI is a visual copilot that eliminates diagramming friction, but human architects must ensure semantic correctness."

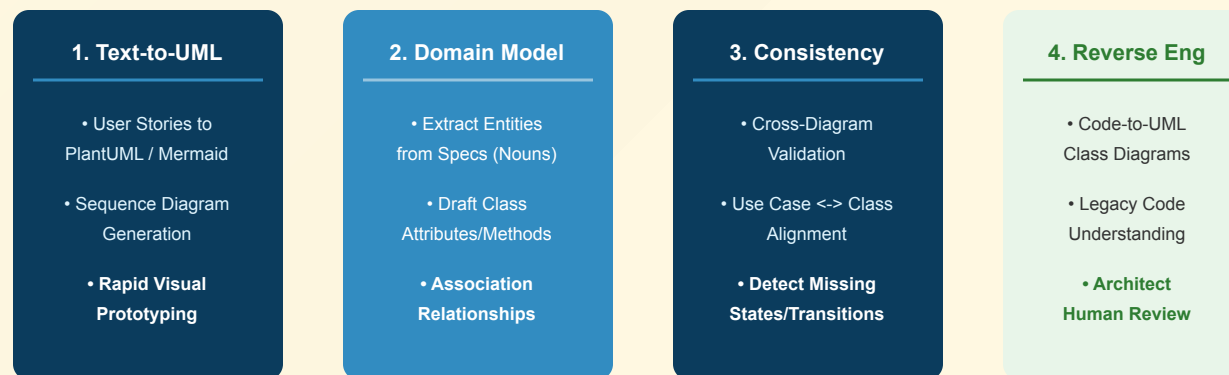
(AI 是一具消除繪圖阻力的視覺副駕駛，但人類架構師必須確保語意的終極正確性。)

”

AI 在系統塑模中的角色：視覺副駕駛 (Visual Copilot)

- 宣告式繪圖革命 (Text-to-Diagram Revolution) :
 - 大型語言模型能直接將非結構化軟體需求轉譯為宣告式文字標記圖表（如 PlantUML、Mermaid.js、Graphviz）。
 - 在數秒內架起自然語言使用者故事與正式圖形架構模型之間的橋樑。

How AI Assists System Modeling (UML & Diagramming Copilot)



- 生產力的大幅躍升：徹底消除惱人的手動排版微調，讓軟體架構師能全神貫注於架構邏輯推理，而非繪圖軟體排版。

AI 在系統塑模中的 4 大核心應用

- **1. 文字直接生成 UML (Text-to-UML Generation) :**
 - 直接根據敏捷使用者故事與 Given-When-Then 驗收準則，自動生成循序圖、類別圖與活動圖標記。
- **2. 領域實體智能萃取 (Domain Entity Extraction) :**
 - 深度語意分析規格需求書，智慧提煉領域名詞（潛在類別、屬性）與動詞（方法操作、關聯）。
- **3. 跨圖表一致性驗證 (Cross-Diagram Consistency Validation) :**
 - 比對檢查使用案例參與者、類別圖與循序圖生命線，偵測命名衝突或未被實作的方法呼叫。
- **4. 程式碼逆向工程塑模 (Code-to-Model Reverse Engineering) :**
 - 讀取現有專案程式碼庫（Java/TypeScript/Python），自動逆向產出類別階層與相依圖，加速新進人員上手。

人機協同防護：AI 塑模風險與最佳實踐

- 未經審查即盲信 AI 塑模的潛在風險：
 - 虛構關聯 (Hallucinated Associations)：幻覺捏造出不存在於業務真實世界中的繼承或組合關聯。
 - 架構過度膨脹 (Architectural Bloat)：硬塞不必要的設計模式（如動輒套用十層工廠裝飾者），違背簡潔原則。
 - 幽靈生命線 (Ghost Lifelines)：在循序圖中憑空捏造不存在的微服務 API 或網路端點。
- 不可動搖的軟體工程黃金守則：

「AI 負責起草繪圖；人類架構師負責檢驗語意！」

(AI Drafts the Diagram; The Human Architect Validates the Semantics!)

軟體架構師必須以批判性工程眼光審視 AI 生成的模型，確保其忠實反映領域真相與架構邊界。

”

觀念檢核測驗 6 (CCQ 6)

當軟體工程團隊導入生成式 AI 輔助自動化產生 UML 圖表時，軟體工程師 / 架構師最核心的職責為何？

- **A.** 拒絕使用任何 AI 工具，堅持純手動編寫每一行繪圖標記語法。
- **B.** 嚴格驗證領域業務語意正確性、結構約束條件與架構簡潔度。
- **C.** 全面廢除傳統程式碼審查（Code Review）與架構設計評審會議。
- **D.** 無條件全盤接受 AI 所生成的所有類別關聯與微服務生命線。



4.8 核心複習與參考文獻

"Models are the lingua franca of software engineering."

(模型，是軟體工程世界的通用語彙。)

”

核心觀念統整：互動填空小測驗

測試你對本章核心概念的掌握度：

1. 在 Rational Software 整合並催生 UML 的「三巨頭」為 Grady Booch、Jim Rumbaugh 與 。
2. 圖用以劃定內部軟體系統與外部夥伴服務雲端之間的營運邊界。
3. 在使用案例塑模中，多個案例必定強制共用的子流程是透過 關係抽離。
4. 將系統劃分為使用者介面、商業邏輯協調與持久化實體的經典架構模式是 。
5. 在 UML 類別圖中，實心黑菱形 () 代表 ，子物件無法脫離父物件獨立存在。
6. 在 UML 類別圖中，空心白菱形 () 代表 ，子物件具備完全獨立的生命週期。
7. UML 狀態機圖之狀態轉移標籤嚴格遵循標準語法：觸發事件 [] / 執行動作。

經典文獻與延伸閱讀 (References)

- 奠基教科書與國際標準規範：

- Sommerville, I. (2016). *Software Engineering* (10th ed.). Chapter 5: System Modeling. Pearson.
- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2nd ed.). Addison-Wesley.
- Fowler, M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.
- Cockburn, A. (2000). *Writing Effective Use Cases*. Addison-Wesley.
- Object Management Group (OMG). (2017). *OMG Unified Modeling Language (OMG UML) Specification*, Version 2.5.1.

- 現代宣告式繪圖標準與 AI 塑模工具：

- PlantUML 開源標準官方網站：plantuml.com
- Mermaid.js JavaScript 宣告式繪圖文件庫：mermaid.js.org